

Guião 07 – IaC com Ansible e Terraform no Proxmox

Infraestruturas de Computação na Nuvem, Aula 07

1 Objetivos

- Configurar o Ansible e definir um *inventory* com as VMs Proxmox como alvos.
- Escrever um *playbook* idempotente que instala e configura o Nginx, com variáveis, templates e handlers.
- Demonstrar concretamente a idempotência, correndo o mesmo *playbook* duas vezes.
- Provisionar uma VM no Proxmox de forma declarativa com Terraform, observando o ciclo *plan/apply/destroy*.
- Distinguir, na prática, provisionamento (Terraform) de gestão de configuração (Ansible).

2 Pré-requisitos e Material

- A VM *icn-lab* (Guião 01), que servirá de **máquina de controlo** – é onde o Ansible e o Terraform correm.
- Pelo menos uma segunda VM no Proxmox, para servir de alvo do Ansible (pode reutilizar uma VM das aulas anteriores, ou criar uma nova).
- Acesso SSH da máquina de controlo para a(s) VM(s) alvo, com privilégios de **sudo**.
- Para a Parte 3 (Terraform): um *API token* do Proxmox com permissões de criação de VMs, fornecido pela docente.

Nota sobre a Parte 3: a criação de VMs por Terraform exige permissões elevadas no Proxmox. Consoante a política do servidor da UC, esta parte pode ser feita por cada aluno com um token restrito a um *pool* próprio, ou realizada como demonstração pela docente. Confirme antes de começar.

3 Introdução

Na aula teórica distinguimos duas camadas complementares de automatização: o **provisionamento** (que recursos existem) e a **gestão de configuração** (o que corre dentro deles). Neste guião vamos exercitar as duas, na ordem inversa da que seria usada em produção – primeiro a configuração, depois o provisionamento – por ser a progressão pedagogicamente mais simples.

Na Parte 1 e 2 usamos o **Ansible**, uma ferramenta *agentless* que comunica por SSH e descreve o estado desejado em YAML. Na Parte 3 usamos o **Terraform** para criar uma VM no Proxmox a partir de código declarativo, fechando o ciclo: Terraform cria a máquina, Ansible configura-a.

Repare que, nas aulas anteriores, criámos VMs manualmente (interface web) e por comandos (*qm*). O que fazemos hoje é o passo seguinte: descrever a infraestrutura em ficheiros versionáveis, em vez de a construir por ações irrepetíveis.

4 Parte 1 – Ansible: preparação

1. **Instalar o Ansible** na máquina de controlo:

```
$ sudo apt update
$ sudo apt install -y ansible
$ ansible --version
```

2. **Configurar acesso SSH por chave** para a(s) VM(s) alvo, evitando pedidos de password a cada execução:

```
$ ssh-keygen -t ed25519 -C "ansible-icn"
$ ssh-copy-id utilizador@<IP-da-VM-alvo>
$ ssh utilizador@<IP-da-VM-alvo> "hostname"
```

3. **Criar a pasta do projeto:**

```
$ mkdir -p ~/ansible-nginx/templates && cd ~/ansible-nginx
```

4. **Criar o *inventory*** `inventory.ini`, com as VMs alvo (substituir pelos IPs reais):

```
[webservers]
vm1 ansible_host=<IP-da-VM-alvo-1> ansible_user=<utilizador>
vm2 ansible_host=<IP-da-VM-alvo-2> ansible_user=<utilizador>

[webservers:vars]
ansible_python_interpreter=/usr/bin/python3
```

Se só tiver uma VM alvo disponível, inclua apenas a linha `vm1`.

5. **Testar a ligação a todos os hosts:**

```
$ ansible -i inventory.ini webservers -m ping
```

Uma resposta `pong` por host confirma que o SSH e o Python estão corretamente configurados.

6. **Observar os *facts* recolhidos automaticamente** de um host:

```
$ ansible -i inventory.ini vm1 -m setup | head -40
```

Localize, entre outros, os seguintes campos:

```
ansible_distribution
ansible_memtotal_mb
ansible_default_ipv4
```

Estes valores podem ser usados em condições e em templates.

5 Parte 2 – Ansible: playbook idempotente

7. **Criar o ficheiro de variáveis** `vars.yml`:

```
site_name: "aula-icn"
http_port: 80
welcome_message: "Servico configurado automaticamente com Ansible"
```

8. **Criar o template Jinja2** templates/index.html.j2. Note o uso de *facts* para incluir informação específica de cada host:

```
<html>
  <head><title>{{ site_name }}</title></head>
  <body>
    <h1>{{ welcome_message }}</h1>
    <p>Servido por: {{ ansible_hostname }}</p>
    <p>Sistema: {{ ansible_distribution }} {{ ansible_distribution_version }}</p>
    <p>Endereco IP: {{ ansible_default_ipv4.address }}</p>
  </body>
</html>
```

O mesmo template gera conteúdo diferente em cada máquina – é aqui que se vê a utilidade dos facts.

9. **Criar o playbook nginx.yml:**

```
---
- name: Instalar e configurar Nginx
  hosts: webserver
  become: true
  vars_files:
    - vars.yml

  tasks:
    - name: Instalar o pacote nginx
      apt:
        name: nginx
        state: present
        update_cache: true

    - name: Implementar pagina de boas-vindas
      template:
        src: templates/index.html.j2
        dest: /var/www/html/index.html
        owner: root
        group: root
        mode: '0644'
      notify: Reiniciar nginx

    - name: Garantir que o serviço está ativo e habilitado
      service:
        name: nginx
        state: started
        enabled: true

  handlers:
    - name: Reiniciar nginx
      service:
        name: nginx
        state: restarted
```

10. **Validar antes de aplicar.** O modo *check* simula a execução sem alterar nada – o equivalente ao *plan* do Terraform:

```
$ ansible-playbook -i inventory.ini nginx.yml --check --diff
```

11. **Executar o playbook** e registrar o resumo final:

```
$ ansible-playbook -i inventory.ini nginx.yml
```

Anote os valores de **changed** e **ok** apresentados no **PLAY RECAP**.

12. **Demonstrar a idempotência.** Execute *exatamente o mesmo comando* uma segunda vez, sem alterar nada:

```
$ ansible-playbook -i inventory.ini nginx.yml
```

Compare o **PLAY RECAP** com o da execução anterior. Desta vez, **changed** deve ser 0: nada foi alterado porque o estado desejado já estava atingido. Repare também que o handler *não* correu – o Nginx não foi reiniciado desnecessariamente.

13. **Verificar o resultado:**

```
$ curl http://<IP-da-VM-alvo-1>
```

Se tiver duas VMs alvo, compare as duas respostas e confirme que os campos preenchidos pelos facts diferem.

14. **Provocar drift e corrigi-lo.** Simule uma alteração manual indevida diretamente na VM alvo:

```
$ ssh utilizador@<IP-da-VM-alvo-1>
$ sudo rm /var/www/html/index.html
$ sudo systemctl stop nginx
$ exit
```

Volte a correr o playbook e observe que ele deteta e repõe automaticamente o estado correto:

```
$ ansible-playbook -i inventory.ini nginx.yml
$ curl http://<IP-da-VM-alvo-1>
```

Este é o mecanismo central da gestão de configuração: convergir sempre para o estado declarado.

15. **Alterar a configuração e observar o handler.** Edite `welcome_message` em `vars.yml` e volte a correr o playbook. Confirme que apenas a task `template` reporta **changed** e que, por isso, o handler é acionado e o Nginx reiniciado.

6 Parte 3 – Terraform: provisionar uma VM no Proxmox

16. **Instalar o Terraform** na máquina de controlo:

```
$ sudo apt install -y wget gpg
$ wget -O- https://apt.releases.hashicorp.com/gpg \
  | sudo gpg --dearmor -o /usr/share/keyrings/hashicorp-archive-keyring.gpg
$ echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] https
  ://apt.releases.hashicorp.com $(lsb_release -cs) main" \
  | sudo tee /etc/apt/sources.list.d/hashicorp.list
$ sudo apt update && sudo apt install -y terraform
$ terraform version
```

17. **Criar a pasta do projeto Terraform:**

```
$ mkdir ~/terraform-proxmox && cd ~/terraform-proxmox
```

18. **Declarar o provider.** Crie `providers.tf`. O Proxmox não tem provider oficial da HashiCorp; usa-se um provider da comunidade (confirme com a docente qual está em uso e a respetiva versão):

```
terraform {
  required_providers {
    proxmox = {
      source = "Telmate/proxmox"
      version = "3.0.1-rc4"
    }
  }
}

provider "proxmox" {
  pm_api_url = var.pm_api_url
  pm_api_token_id = var.pm_api_token_id
  pm_api_token_secret = var.pm_api_token_secret
  pm_tls_insecure = true
}
```

19. **Declarar as variáveis.** Crie `variables.tf`, mantendo os segredos fora do código:

```
variable "pm_api_url" {
  type = string
}

variable "pm_api_token_id" {
  type = string
  sensitive = true
}

variable "pm_api_token_secret" {
  type = string
  sensitive = true
}

variable "target_node" {
  type = string
}

variable "vm_template" {
  type = string
}
```

Os valores concretos vão num ficheiro `terraform.tfvars` que **nunca deve ser commitado** num repositório:

```
pm_api_url = "https://<IP-do-servidor>:8006/api2/json"
pm_api_token_id = "<utilizador>@pve!<nome-do-token>"
pm_api_token_secret = "<segredo-do-token>"
target_node = "<nome-do-node>"
vm_template = "<nome-do-template>"
```

Crie também um `.gitignore` com `terraform.tfvars` e `*.tfstate*`, pelas razões discutidas na teoria.

20. **Declarar o recurso.** Crie `main.tf`:

```
resource "proxmox_vm_qemu" "vm_iac" {
  name = "icn-terraform"
  target_node = var.target_node
  clone = var.vm_template
  full_clone = true

  cores = 2
  memory = 2048

  network {
    id = 0
    model = "virtio"
    bridge = "vbr0"
  }
}
```

Nota: a sintaxe exata dos blocos varia entre versões do provider. Se o `terraform validate` acusar erros, consulte a documentação da versão instalada – este ajuste faz parte do exercício.

21. **Inicializar e validar:**

```
$ terraform init
$ terraform validate
$ terraform fmt
```

22. **Executar o plan.** Este é o passo conceitualmente mais importante: mostra o que *seria* feito, sem fazer nada:

```
$ terraform plan
```

Leia com atenção o resumo final (Plan: 1 to add, 0 to change, 0 to destroy) e registre-o.

23. **Aplicar:**

```
$ terraform apply
```

Confirme na interface web do Proxmox que a VM foi efetivamente criada.

24. **Inspecionar o estado registrado:**

```
$ terraform state list
$ terraform show
```

25. **Observar a detecção de drift.** Na interface web do Proxmox, altere manualmente a memória da VM criada (ex. para 1024 MB). De volta ao terminal:

```
$ terraform plan
```

O Terraform detecta a divergência entre o estado real e o declarado, e propõe repô-la. Este é o comportamento discutido na teoria a propósito do *state file*.

26. **Destruir a infraestrutura criada:**

```
$ terraform destroy
```

Confirme no Proxmox que a VM desapareceu.

7 Entregável / Questões de Verificação

- a) Submeta `nginx.yml`, `vars.yml`, o template Jinja2 e o `inventory.ini` (com IPs anonimizados, se necessário).
- b) Submeta os dois `PLAY RECAP` obtidos (primeira e segunda execução do playbook). Explique a diferença nos valores de `changed` recorrendo ao conceito de idempotência.
- c) Porque é que o handler não foi executado na segunda passagem, mas foi executado quando alterou o `welcome_message`?
- d) Descreva o que aconteceu quando apagou manualmente o `index.html` e voltou a correr o playbook. Que problema operacional (visto na teoria) é assim resolvido?
- e) Submeta o resultado de `terraform plan` antes do `apply`. Que vantagem tem esta separação entre planear e aplicar, comparada com executar um script diretamente?
- f) Alterou a memória da VM manualmente no Proxmox e o Terraform detetou-o. Que ficheiro lhe permitiu detetar essa divergência, e porque é que esse ficheiro deve ser tratado como um segredo?
- g) Neste guião usou duas ferramentas distintas. Qual delas fez *provisionamento* e qual fez *gestão de configuração*? Descreva, em duas ou três frases, como as encadearia num fluxo de trabalho real.
- h) O ficheiro `terraform.tfvars` não deve ser commitado. Que alternativas existem para gerir segredos num projeto de IaC versionado em Git?